



Instituto Federal de Mato Grosso do Sul
Disciplina: Linguagem de Programação III
Professora: Milena Alegre

LISTA 01

Orientações gerais

Esta lista deverá ser desenvolvida individualmente. Todas as atividades práticas são voltadas ao backend com Node.js e Express. Não é necessário criar páginas HTML, CSS ou qualquer interface front-end.

Todos os endpoints solicitados deverão ser testados no Postman. Quando solicitado, inclua no README prints que mostrem claramente o método HTTP, a URL utilizada, o Body em JSON (quando houver), o status da resposta e o resultado retornado pelo servidor.

Parte 1 - Conceitos iniciais de Node.js, NPM e Express

1. Explique com suas palavras o que é Node.js e qual é sua função em uma aplicação web.
2. Qual é a diferença principal entre executar JavaScript no navegador e executar JavaScript com Node.js?
3. Explique o que é NPM.
4. Explique a função do arquivo package.json.
5. Explique por que a pasta node_modules normalmente não é enviada para o GitHub.
6. Crie um novo projeto Node.js utilizando npm init -y.
7. Instale o Express no projeto.
8. Crie o arquivo server.js e importe/configure o Express.
9. Configure o servidor para escutar na porta 3000.
10. Crie a rota GET / que retorne uma mensagem informando que a API está funcionando.

Teste GET / no Postman e registre o resultado no README.

Parte 2 - Rotas, requisições e respostas

11. Explique o que é uma rota (endpoint) em uma API.
12. Explique a função de req em uma rota Express.
13. Explique a função de res em uma rota Express.
14. Explique a diferença entre res.send() e res.json().



15. Explique o significado de API REST ou RESTful dentro do conteúdo trabalhado.
16. Associe cada método HTTP à sua finalidade: GET, POST, PUT e DELETE.
17. Explique a diferença entre req.body e req.params.
18. Explique para que serve `app.use(express.json())`.

Parte 3 - Estrutura inicial de dados

19. Crie inicialmente um array chamado jogos contendo pelo menos 3 objetos.
Cada jogo deverá possuir: id, titulo, genero, ano e nota.
20. Crie GET /jogos para retornar todos os jogos.
21. Teste GET /jogos no Postman.
22. Crie GET /jogos/:id para buscar apenas um jogo pelo ID.
23. Utilize req.params.id para capturar o ID informado na URL.
24. Utilize find() para localizar o jogo.
25. Caso o jogo não exista, retorne status 404 e uma mensagem em JSON.
26. Teste no Postman um ID existente e um ID inexistente.
Inclua os dois testes no README.

Parte 4 - POST: criação de dados

27. Crie POST /jogos para cadastrar um novo jogo.
28. O Body deverá ser enviado pelo Postman no formato JSON.
Exemplo: `{"titulo": "Minecraft", "genero": "Aventura", "ano": 2011, "nota": 9}`
29. Capture os dados utilizando req.body.
30. Crie o ID do novo jogo automaticamente.
31. Adicione o novo objeto ao array utilizando push().
32. Caso titulo ou genero não sejam enviados, retorne status 400.
33. Quando o cadastro for realizado corretamente, utilize status 201.
34. Teste no Postman um POST válido.
35. Teste no Postman um POST com dados obrigatórios ausentes.
Os prints devem mostrar o Body enviado e a resposta recebida.

Parte 5 - PUT: atualização de dados

36. Crie PUT /jogos/:id.



37. Localize o jogo utilizando o ID recebido por req.params.
38. Permita alterar título, gênero, ano e nota por meio do Body JSON.
39. Caso o jogo não exista, retorne status 404.
40. Retorne o objeto atualizado após a alteração.
41. Teste no Postman uma atualização válida.
42. Teste uma tentativa de atualização utilizando um ID inexistente.
43. Explique por que o PUT é diferente do POST.

Parte 6 - DELETE: exclusão de dados

44. Crie DELETE /jogos/:id.
45. Utilize findIndex() para localizar a posição do jogo no array.
46. Utilize splice() para remover o jogo.
47. Caso o ID não exista, retorne status 404.
48. Retorne uma mensagem confirmando a exclusão.
49. Teste o DELETE no Postman.
50. Explique por que, neste caso, o DELETE não precisa receber Body.
51. Depois da exclusão, faça um GET /jogos e comprove que o item foi removido.

Parte 7 - Rota especial e manipulação de arrays

52. Crie GET /jogos/melhores.
53. Essa rota deverá retornar apenas jogos com nota maior ou igual a 8.
54. Utilize filter() para realizar a seleção.
55. Teste a rota no Postman.
56. Explique a diferença entre find(), findIndex() e filter().
57. Explique a diferença entre push() e splice().

Parte 8 - JSON: teoria e conversão

58. Explique o que é JSON.
59. Explique a função de JSON.parse().
60. Explique a função de JSON.stringify().
61. Explique por que um arquivo JSON armazenado no disco precisa ser lido como texto antes de ser manipulado como objeto/array JavaScript.



62. Explique a finalidade dos parâmetros null, 2 em JSON.stringify(dados, null, 2).
63. Identifique pelo menos três regras de sintaxe de um JSON válido.
64. Explique a diferença entre um objeto JavaScript em memória e o texto armazenado em um arquivo .json.

Parte 9 - Persistência em arquivo JSON

65. Crie uma pasta dados e, dentro dela, o arquivo jogos.json.
66. Transfira os jogos iniciais para jogos.json.
67. Leia o conteúdo de jogos.json utilizando o módulo fs.
68. Converta o conteúdo lido utilizando JSON.parse().
69. Faça GET /jogos retornar os dados lidos do arquivo, em vez de depender somente de um array criado diretamente no server.js.
70. No POST /jogos, leia o arquivo, faça JSON.parse(), adicione o novo jogo com push(), converta novamente com JSON.stringify() e grave o arquivo.
71. Depois de cadastrar um jogo pelo Postman, reinicie o servidor e comprove que o jogo continua cadastrado.
72. Explique por que os dados agora permanecem após o servidor ser desligado.
73. Adapte o PUT para que a alteração também seja salva em jogos.json.
74. Adapte o DELETE para que a exclusão também seja salva em jogos.json.
75. Teste novamente GET, POST, PUT e DELETE no Postman após implementar a persistência.

Os dados do arquivo jogos.json deverão realmente mudar após as operações.

Parte 10 - Manipulação de arquivo TXT e histórico

76. Crie o arquivo historico.txt.
77. Sempre que um jogo for cadastrado, acrescente uma linha no histórico.
Formato sugerido: JOGO CADASTRADO: Minecraft
78. Sempre que um jogo for atualizado, acrescente uma linha informando a atualização.
79. Sempre que um jogo for removido, acrescente uma linha informando a remoção.
80. Utilize appendFile ou appendFileSync para acrescentar dados sem apagar o conteúdo anterior.
81. Crie GET /historico para ler e retornar o conteúdo de historico.txt.



- 82. Teste GET /historico no Postman.
- 83. Explique a diferença entre writeFile e appendFile.
- 84. Explique o que pode acontecer com o conteúdo anterior de um arquivo quando writeFile é utilizado sobre um arquivo que já existe.
- 85. Explique a finalidade de readFile.

Parte 11 - Exclusão de arquivos e módulo fs

- 86. Explique a função de fs.unlink().
- 87. Explique o que acontece quando fs.unlink() é usado para remover um arquivo.
- 88. Associe as operações abaixo aos métodos de arquivo correspondentes: criar/escrever, ler, acrescentar e excluir.
- 89. Explique o que significa o erro ENOENT.
- 90. Cite uma situação do projeto em que ENOENT poderia ocorrer.

Parte 12 - path e caminhos de arquivos

- 91. Explique para que serve o módulo path do Node.js.
- 92. Explique por que escrever caminhos manualmente pode causar problemas entre Windows, Linux e macOS.
- 93. Utilize path.join() para montar o caminho de jogos.json.
- 94. Utilize path.join() para montar o caminho de historico.txt.
- 95. Explique a vantagem de utilizar path.join() no projeto.

Parte 13 - Tratamento de erros

- 96. Explique a função do bloco try/catch.
- 97. Implemente tratamento de erro em pelo menos uma operação de leitura de arquivo.
- 98. Implemente tratamento de erro em pelo menos uma operação de escrita de arquivo.
- 99. Caso ocorra um erro interno inesperado em uma rota, retorne uma resposta de erro apropriada ao cliente.
- 100. Teste uma situação de erro controlado e descreva no README o que aconteceu.

Parte 14 - Síncrono, assíncrono e Event Loop

- 101. Explique a diferença entre uma operação síncrona e uma operação assíncrona.



102. Explique o que acontece com o servidor quando uma operação síncrona demorada bloqueia a execução.
103. Explique, de acordo com o conteúdo trabalhado, por que operações assíncronas são preferíveis em rotas de servidor.
104. Explique o que é uma Promise.
105. Explique a função de async.
106. Explique a função de await.
107. Compare readFileSync com readFile.
108. Se utilizar a versão assíncrona no projeto, envolva a operação em try/catch.

Parte 15 - Middlewares

109. Explique o que é um middleware no Express.
110. Explique por que express.json() pode ser considerado um middleware.
111. Cite duas outras responsabilidades que um middleware pode assumir em uma aplicação.
112. Explique em que momento o middleware atua no fluxo requisição -> rota -> resposta.

Parte 16 - Sessões e Cookies - SOMENTE TEORIA

Nesta parte não é necessário instalar bibliotecas nem implementar login, sessão ou cookie. Responda somente com base nos conceitos estudados.

113. Explique por que o protocolo HTTP é considerado stateless.
114. Explique o que é um Cookie.
115. Explique o que é uma Sessão.
116. Onde os dados de um Cookie ficam armazenados?
117. Onde os dados de uma Sessão ficam armazenados?
118. Explique como Cookie e Sessão podem trabalhar juntos para reconhecer um usuário entre diferentes requisições.
119. Cite um exemplo de uso adequado para Cookie.
120. Cite um exemplo de uso adequado para Sessão.
121. Explique, de forma conceitual, o que é Session ID.

Parte 17 - Testes obrigatórios no Postman

122. Crie no Postman uma requisição para GET /.



123. Crie uma requisição para GET /jogos.
124. Crie uma requisição para GET /jogos/:id.
125. Crie uma requisição para POST /jogos com Body JSON.
126. Crie uma requisição para PUT /jogos/:id com Body JSON.
127. Crie uma requisição para DELETE /jogos/:id.
128. Crie uma requisição para GET /jogos/melhores.
129. Crie uma requisição para GET /historico.
130. Para cada operação principal, registre no README pelo menos um print que mostre a execução.
131. Em POST e PUT, o print deverá mostrar o Body JSON utilizado.
132. Inclua pelo menos um teste que resulte em status 404.
133. Inclua pelo menos um teste que resulte em status 400.
134. Inclua pelo menos um teste que resulte em status 201.
135. Não será considerado suficiente entregar apenas o código sem evidência de execução das rotas.

Parte 18 - Organização e entrega

136. Organize o projeto de forma clara, separando os arquivos de dados do arquivo principal do servidor.

Estrutura mínima sugerida:

server.js

package.json

dados/jogos.json

dados/historico.txt

README.md

137. Não envie a pasta node_modules para o repositório.
138. Inclua no README os comandos necessários para instalar as dependências e iniciar o servidor.
139. Inclua no README as respostas das questões teóricas.
140. Inclua no README os prints solicitados dos testes no Postman.
141. Envie o link do repositório no GitHub conforme orientação da professora.



Critérios de avaliação

Critério	Valor
Servidor Node/Express e organização inicial	1,0
Rotas GET e busca por ID	1,0
POST, PUT e DELETE	2,0
Validações, status HTTP e rotas especiais	1,0
Persistência com JSON	1,5
Manipulação de arquivos TXT / fs	1,0
path, tratamento de erros e conceitos assíncronos	1,0
Questões teóricas, incluindo Sessões e Cookies	0,5
Testes e evidências no Postman / README	1,0
TOTAL	10,0

Importante: toda a parte prática desta lista deverá ser executada no backend e testada pelo Postman.

Bom Trabalho! 😊